

SQLインジェクション

——攻撃の仕組みから対策まで

田中博悠 / 2026年7月

対象者

- SQLの基本文法（`SELECT` / `INSERT` / `WHERE`）を読んだことがある人
- Web開発でDBを使ったことがある人（言語・フレームワークは問わない）
- 「SQLインジェクション」という言葉は知っているが、手を動かしたことはない人
- ZAPで脆弱性診断をしたことはあるが、検出後に何が起きるかを深く見たい人

SQL文法そのものの解説はしません。攻撃者視点で何が漏れるかと、その対策に集中します。

はじめに



このスライドと配布するサンドボックスは学習目的専用です

- 許可のないシステムへの攻撃は不正アクセス禁止法等の違反です
- PoCは必ず自分のPC上でローカルに動かした環境に対してのみ実行してください

アジェンダ

1. 講義の安全範囲とZAPの役割
2. SQLインジェクションの仕組み
3. 基本編：ログインバイパス / UNION情報漏洩
4. 発展編：ブラインド / スタック型 / セカンドオーダー
5. 実DB編：MySQLで見る方言差と漏洩
6. NoSQL編：MongoDB NoSQL injection
7. 総合演習：コードを見ずに探す
8. 対策：プリペアドステートメントほか
9. まとめ

講義のゴール

「検出」から「被害」までをつなげる

通常の診断講義では、ZAPなどで脆弱性を見つけて対策を確認することが多い。
今回はその間にある、攻撃者視点の流れを重点的に見る。

1. ZAPで脆弱性の疑いを見つける
2. Request / Response / Evidence を読む
3. 手動ペイロードで再現する
4. 架空の認証情報・顧客情報が漏洩するところまで確認する
5. 修正版で同じ攻撃が無効化されることを確認する

ZAPはDB種別まで分かる？

結論：推測材料は出せるが、常に断定できるわけではない

- ZAPのSQL Injection alertは、MySQL / PostgreSQL / SQLite / Oracle / MSSQLなど複数DBを対象技術として扱う
- Technology Detectionはレスポンス内の証拠から技術を推測する
- ただしDBが裏側に隠れている場合、画面上の情報だけでDB種別を断定できないことも多い

見るべきもの：

- SQLエラーの文面
- コメント記法（`--` / `#` など）
- 時間差関数（`SLEEP()` など）
- メタデータ参照（`information_schema` など）

ZAPから手動攻撃につなげる

ZAPのAlertは入口

1. Spiderで画面とパラメータを集める
2. Active Scanで `SQL Injection` のAlertを確認する
3. Alertsの `Evidence`、HistoryのRequest/Responseを見る
4. Requesterやブラウザでペイロードを少しずつ変える
5. 情報漏洩が再現できるかを確認する

ZAPの検出結果だけで「攻撃できる」と判断しない。
逆にZAPで検出されないから安全とも判断しない。

演習の進め方（4ステップ）

各PoCはこの4ステップで進めます。攻撃して終わりにしないのがポイント。

1. 🔍 脆弱なコードを読む（ `app.py` ） → どこが危ないか予想する
2. ⚡ 攻撃してみる → ペイロードを入力し、突破できることを確認する
3. 🛠️ 自分で修正する → `app.py` の該当箇所をプレースホルダに書き換える
4. ✅ 再攻撃して防御を確認する → 同じペイロードが効かなくなることを確認する

答え合わせ用に `secure_app.py` を用意していますが、
まず自分で直してから見るとより理解が深まります

1. SQLインジェクションとは

定義

アプリケーションがユーザーの入力をそのままSQL文に組み込んでしまうことで、攻撃者が意図しないSQLを実行できてしまう脆弱性

できてしまうこと

- 認証の回避（ログインバイパス）
- 他人のデータの閲覧・改ざん・削除
- データベース全体の抽出
- 場合によってはOS上のコマンド実行まで拡大することも

OWASP Top 10 での位置づけ

- 長年 Injection カテゴリの中心的存在
- 2021年版では A03:2021 – Injection に統合
- 依然として実際のインシデントで多発する脆弱性

「古い脆弱性」ではなく「今も現役の脆弱性」

2. なぜ起こるのか

文字列結合でSQLを組み立てるコード

```
username = request.form["username"]
password = request.form["password"]

query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
cur.execute(query)
```

- `username` や `password` にSQLの構文として意味を持つ文字（`'` や `--` など）を混ぜられると、SQL文の意味が変わってしまう

文字列結合の危険性（図解）

本来のSQL:

```
SELECT * FROM users WHERE username = '[入力]' AND password = '[入力]'
```

username に admin' -- を入力すると...

```
SELECT * FROM users WHERE username = 'admin' --' AND password = '...'
                                     ↑ ここから後ろはコメント化
```

→ パスワードチェックの部分が丸ごと無視される

3. 基本編：ログインバイパス

典型的なペイロード

ユーザー名に入力	効果
admin' --	パスワード条件をコメントアウト
admin' #	MySQLでの同上（# がコメント）
' OR '1'='1' --	常に真になる条件を追加

'1'='1' は常に真 → WHERE 条件全体が真になり、最初のレコードでログイン成立

PoC①：ログインバイパスをやってみよう

フォルダ： `01_login_bypass/`

```
cd 01_login_bypass && pip install -r requirements.txt && python app.py  
# http://127.0.0.1:5001
```

1. `app.py` の文字列結合部分を確認する
2. ユーザー名 `admin' --` / パスワード空欄でログイン → 突破できることを確認
3. `app.py` を `?` プレースホルダに書き換えてみる
4. 同じ入力でログインが失敗することを確認（答え合わせ： `secure_app.py` ）

🎯 PoC①で理解してほしいこと

- 文字列結合だと、入力に含まれる `'` がSQLの構文の一部として解釈されてしまう
→ `--` や `OR '1'='1'` で条件を自由に書き換えられる
- プレースホルダ（`?`）を使うと、入力値は常に「単なる値」として渡される
→ `'` や `--` が入っていても構文としては解釈されない
- この「文字列結合 → プレースホルダ」というパターンは、以降のPoCでも繰り返し出てくる

4. 基本編：UNIONベースの攻撃

UNION SELECT とは

2つの `SELECT` の結果を縦に結合するSQL構文。

検索結果として表示される画面に、別のテーブルの内容を紛れ込ませることができる。

攻撃の手順

1. `ORDER BY` などでもラム数を特定する
2. `UNION SELECT` で列数・型を一致させる
3. 抜き出したいテーブル（例： `users` ）を指定する

UNION攻撃の実例

脆弱な検索クエリ（商品名検索、3カラムを表示）：

```
SELECT id, name, price FROM products WHERE name LIKE '%[入力]%'
```

攻撃者の入力：

```
%' UNION SELECT id, username, password FROM users --
```

→ 商品一覧の中にユーザー名とパスワードが混ざって表示される

顧客情報を抜く例：

```
%' UNION SELECT id, email, last_order_total FROM customers --
```

→ 商品一覧の中に顧客メールアドレスと注文金額が混ざって表示される

PoC② : UNIONインジェクションをやってみよう

フォルダ: `02_union_based/`

```
cd 02_union_based && pip install -r requirements.txt && python app.py
# http://127.0.0.1:5002
```

1. `app.py` の検索 (`LIKE`) 部分の文字列結合を確認する
2. `%' UNION SELECT id, username, password FROM users --` で `users` テーブルを抽出
3. `%' UNION SELECT id, email, last_order_total FROM customers --` で顧客情報を抽出
4. `app.py` をプレースホルダに書き換えてみる
5. 同じペイロードでヒット件数が0件になることを確認 (答え合わせ: `secure_app.py`)

🎯 PoC②で理解してほしいこと

- 検索（`LIKE`）のように値の一部を組み立てる処理でも、文字列結合なら同じく危険
- `UNION SELECT` は「列数と型を合わせれば別テーブルの内容を差し込める」という仕組み
- プレースホルダに渡す値は `%keyword%` のように組み立てた後の文字列でもよい
→ 大事なのは「SQL文の構造」に手を加えられないようにすること

エラーベースの補助テクニック

- DBがエラーメッセージをそのまま画面に返す設計だと、エラー文からテーブル名・カラム名・DB種別が漏れる
- 例： `' AND 1=CONVERT(int, (SELECT TOP 1 table_name FROM information_schema.tables)) --`
- カラム数の特定にも `ORDER BY 1,2,3...` を使ってエラーの有無で判定できる

⚠ 本番環境で詳細なDBエラーを表示するのはそれ自体が情報漏洩

5. 発展編：ブラインドSQLインジェクション

画面に結果が出ない場合はどうする？

エラーもデータも表示されない場合でも、
真偽（Yes/No）の違いさえ観測できれば情報を抜き出せる

- Boolean-based：レスポンスの違い（表示 or 非表示）で判定
- Time-based：応答時間の違い（遅延の有無）で判定

Boolean-based Blind の例

正常な注文照会：

```
SELECT * FROM orders WHERE id = [入力]
```

真になる入力 → 結果が表示される：

```
1 AND 1=1
```

偽になる入力 → 結果が表示されない：

```
1 AND 1=2
```

この「表示/非表示」の差分を1文字ずつ繰り返すことで、
`SUBSTR()` などと組み合わせてDBの内容を1文字ずつ確定させられる

Time-based Blind の例

画面に差が出ない場合は、応答時間の差を使う

```
1 AND (CASE WHEN (SUBSTR((SELECT password FROM users LIMIT 1),1,1)='a') THEN SLEEP(5) ELSE 0 END)
```

- 条件が真 → 5秒待たされる
- 条件が偽 → 即座に応答が返る

→ 画面に何も表示されなくても、時間差だけで真偽が分かる

PoC③：ブラインドSQLインジェクションをやってみよう

フォルダ： `03_blind_injection/`

```
cd 03_blind_injection && pip install -r requirements.txt && python app.py  
# http://127.0.0.1:5003
```

1. `app.py` の数値パラメータの文字列結合部分を確認する
2. `/track?order_id=` に Boolean-based / Time-based の両方を試す → 突破確認
3. `python extract_password.py` で1文字ずつの抽出を自動化して見る
4. `app.py` を `int()` チェック+プレースホルダに書き換えてみる
5. 同じペイロードがエラーになり実行されないことを確認（答え合わせ： `secure_app.py` ）

🎯 PoC③で理解してほしいこと

- 画面に何も表示されなくても、Yes/No・応答時間の差だけで情報を1文字ずつ抜き出せる
- 数値パラメータだからと「」で囲まなくても、文字列結合であれば同じく危険
- 対策は「型を検証する」と「プレースホルダを使う」の両方を組み合わせるのが効果的

6. 発展編：スタック型SQLインジェクション

Stacked Queries とは

1回のリクエストで複数のSQL文をセミコロン区切りで連続実行させる攻撃

```
note') ; DROP TABLE notes; --
```

- `SELECT` だけでなく `INSERT` / `UPDATE` / `DELETE` / `DROP` まで実行できてしまう
- ドライバや設定によって複数文実行が許可されている場合に成立する

発展編：セカンドオーダー SQLインジェクション

「今」ではなく「後で」爆発する攻撃

1. ユーザー登録時、ユーザー名はプレースホルダで安全に保存される
2. しかし別の機能（例：管理者向けレポート）が、
保存済みの値をそのまま文字列結合でSQLに使ってしまう
3. 登録時には無害に見えた値が、後から実行されるSQLで爆発する

入力時点だけでなく、値を使うすべての場所で対策が必要という教訓

PoC④：スタック型 & セカンドオーダーをやってみよう

フォルダ： `04_advanced_second_order/`

```
cd 04_advanced_second_order && pip install -r requirements.txt && python app.py  
# http://127.0.0.1:5004
```

1. `app.py` の `executescript()` と `/admin/report` の文字列結合部分を確認する
2. `/note` でスタック型、`/register` → `/admin/report` でセカンドオーダーを試す → 突破確認
3. `app.py` を `execute()` + プレースホルダに書き換えてみる（脆弱ポイントは2箇所）
4. 同じ手順で攻撃が通らなくなることを確認（答え合わせ： `secure_app.py` ）

🎯 PoC④で理解してほしいこと

- 複数文を連続実行できる実装（`executescript()` 等）は、
 `INSERT` だけを想定していても `DROP` / `UPDATE` まで実行されてしまう
- 「すでにDBに保存された値だから安全」という思い込みが最も危険
 → 対策は「入力を受け取る場所」だけでなく値を使うすべての場所に必要
- ここまでの4つのPoCは形は違うが、根本原因はすべて同じ「文字列結合」

7. 実DB編：MySQLで見るSQLインジェクション

SQLiteとの違いも見せる

SQLiteのPoCは準備が軽い一方、実DBらしい差分は見えにくい。
MySQL版では以下を確認する。

- `--` コメントの後ろに空白が必要
- エラー文からDB種別や構文のヒントが漏れることがある
- `information_schema` のようなメタデータ参照がある
- `SLEEP()` など時間差攻撃に使われる関数が存在する

PoC⑤ : MySQL実DBで情報漏洩を再現する

フォルダ : 05_mysql_realdb/

```
cd sandboxes/poc_sql_injection
docker compose up --build
# 脆弱版 http://127.0.0.1:5005
# 修正版 http://127.0.0.1:5015
```

1. ZAPで `http://127.0.0.1:5005/` をSpider/Active Scanする
2. `'` や `ORDER BY` でエラーとカラム数を確認する
3. `zzz%' UNION SELECT id, email, last_order_total FROM customers --` を入力する
4. 商品検索画面に架空顧客のメールアドレス・注文金額が漏れることを確認する
5. 修正版 `5015` で同じ攻撃が効かないことを確認する

8. NoSQL injection

SQL以外にも「入力がクエリ構造になる」と危険

MongoDBではクエリがJSON風のオブジェクトで表現される。

アプリがリクエストJSONをそのまま検索条件に渡すと、値ではなく条件式を混入される。

```
{  
  "username": {"$ne": null},  
  "password": {"$ne": null}  
}
```

`$ne` は「等しくない」という条件。

文字列として扱うべき入力が条件オブジェクトとして解釈されると、認証をすり抜ける。

PoC⑥ : MongoDB NoSQL injection

フォルダ : 06_mongodb_nosql/

```
cd sandboxes/poc_sql_injection
docker compose up --build
# 脆弱版 http://127.0.0.1:5006
# 修正版 http://127.0.0.1:5016
```

攻撃例 :

```
curl -s -X POST http://127.0.0.1:5006/login \
-H 'Content-Type: application/json' \
-d '{"username":{"$ne":null},"password":{"$ne":null}}'
```

→ パスワードを知らなくても最初に一致したユーザーでログイン成功する

🎯 PoC⑤・⑥で理解してほしいこと

- SQLiteだけでなく、実DBではコメント記法・エラー・関数・メタデータに差がある
- ZAPは検出の入口。DB種別や攻撃可否は手動確認で詰める
- NoSQLでも「ユーザー入力クエリ構造に混ざる」と同じ種類の問題が起きる
- SQLならプレースホルダ、MongoDBなら型検証・スキーマ検証・演算子オブジェクト拒否が重要

9. 見つけ方：ブラックボックス探索の型

闇雲に攻撃しない

まずアプリの入口を洗い出す。

観察対象	見ること
フォーム	入力名、必須項目、エラー表示
URL	id / q / filter などのパラメータ
JSON API	値が文字列か、オブジェクトも受けるか
レスポンス	表示差分、エラー、応答時間
保存機能	入力した値が別画面で再利用されるか

ZAPのSpider/Historyで入口を集め、怪しい順に手動確認する。

見つけ方：SQLiの切り分け

小さく差分を見る

1. 通常入力で基準レスポンスを確認する
2. ' や " でエラー・表示差分を見る
3. Boolean条件で真偽の差を見る
 - 1 AND 1=1
 - 1 AND 1=2
4. ORDER BY で列数を推測する
5. UNION SELECT で表示可能な列を探す
6. 結果が出ない場合は応答時間を見る

本講義ではローカル教材だけで実施する。実在サイトには絶対に試さない。

見つけ方：NoSQL / セカンドオーダー

SQLらしく見えない場所も疑う

NoSQL injection :

- JSON APIで文字列の代わりにオブジェクトを送れるか
- `{"$ne": null}` や `{"$gt": 0}` のような条件が値として扱われていないか
- レスポンス件数・認証結果が変わるか

セカンドオーダー :

- 登録画面では何も起きない値が、管理画面・レポート・検索で後から効かないか
- 「保存された値だから安全」という思い込みがないか

PoC⑦：総合演習 Black-box Challenge

フォルダ： `07_challenge_blackbox/`

```
cd sandboxes/poc_sql_injection
docker compose up --build
# http://127.0.0.1:5007
```

ルール：

- コードを見ずに、ZAP・ブラウザ・curlで探す
- 対象は `127.0.0.1:5007` のみ
- 見つけたら「場所」「脆弱性の種類」「再現手順」「漏れた/変わった情報」を記録する

PoC⑦で探すもの

これまでの脆弱性を全部混ぜてある

種類	ありそうな場所の例
ログインバイパス	認証フォーム
UNION情報漏洩	商品検索・一覧
Boolean/Time-based Blind	注文照会などの数値パラメータ
Stacked Queries	投稿・メモ・保存フォーム
セカンドオーダー	登録後に別画面で使われる値
NoSQL injection	JSON API

講義では、最初に答えを見ずに探索し、最後に講師が答え合わせする。

10. 対策①：プレースホルダ（プリペアドステートメント）

最も重要かつ最も効果的な対策

```
#  脆弱
query = f"SELECT * FROM users WHERE username = '{username}'"
cur.execute(query)

#  安全
cur.execute("SELECT * FROM users WHERE username = ?", (username,))
```

- 値はSQL構文とは別チャンネルでDBに渡される
- 入力に `'` や `--` が入っていても単なる文字列として扱われる

対策②：ORM / クエリビルダの活用

```
# SQLAlchemy の例（内部でプレースホルダが使われる）  
User.query.filter_by(username=username).first()
```

- 生SQLを手書きしない分、事故が起きにくい
- ただし `raw()` / `text()` などでは生SQLを書ける逃げ道は残るので注意

対策③：入力値の検証・最小権限

入力値検証（バリデーション）

- 想定外の文字種・長さを拒否する（多層防御の一つ、これ単体では不十分）
- MongoDBでは `username` / `password` などを文字列型に限定し、
`{"$ne": null}` のような演算子オブジェクトを拒否する

最小権限の原則

- アプリ用DBユーザーに必要最小限の権限だけ与える
- `DROP` / `information_schema` への参照などは必要なければ禁止

エラーメッセージ

- 本番環境では詳細なDBエラーを画面に出さない

対策④：多層防御（Defense in Depth）

層	対策
コード	プレースホルダ / ORM
DB	最小権限アカウント、ストアドプロシージャ
ネットワーク	WAF による既知パターンの検知・遮断
運用	ログ監視、依存ライブラリの更新、定期的な脆弱性診断

「1つ対策すれば終わり」ではなく、複数の層を重ねる

まとめ

フェーズ	手法	対策
基本	ログインバイパス / UNION	プレースホルダ
発展	ブラインド (Boolean/Time)	詳細エラー非表示・最小権限
発展	スタック型・セカンドオーダー	使う場所すべてでプレースホルダ
実DB	MySQL方言差・エラー・メタデータ	パラメータ化・詳細エラー非表示
NoSQL	MongoDB条件オブジェクト混入	型検証・スキーマ検証・演算子拒否
総合演習	複数箇所の混在	入口ごとの洗い出し・分類・再現
全体	—	多層防御・入力値検証・WAF・監視

SQLインジェクションは古典的だが今も現役の脆弱性。
「どこかで文字列結合していないか」を常に疑う姿勢が最大の防御。

参考リンク

- [OWASP SQL Injection](#)
- [OWASP SQL Injection Prevention Cheat Sheet](#)
- [PortSwigger Web Security Academy - SQL injection](#)
- [ZAP SQL Injection Alert](#)
- [ZAP Technology Detection](#)

Q&A

ご質問はありますか？

配布したサンドボックスは、講座終了後もローカルで復習に使ってください。